Data Warehousing with Snowflake

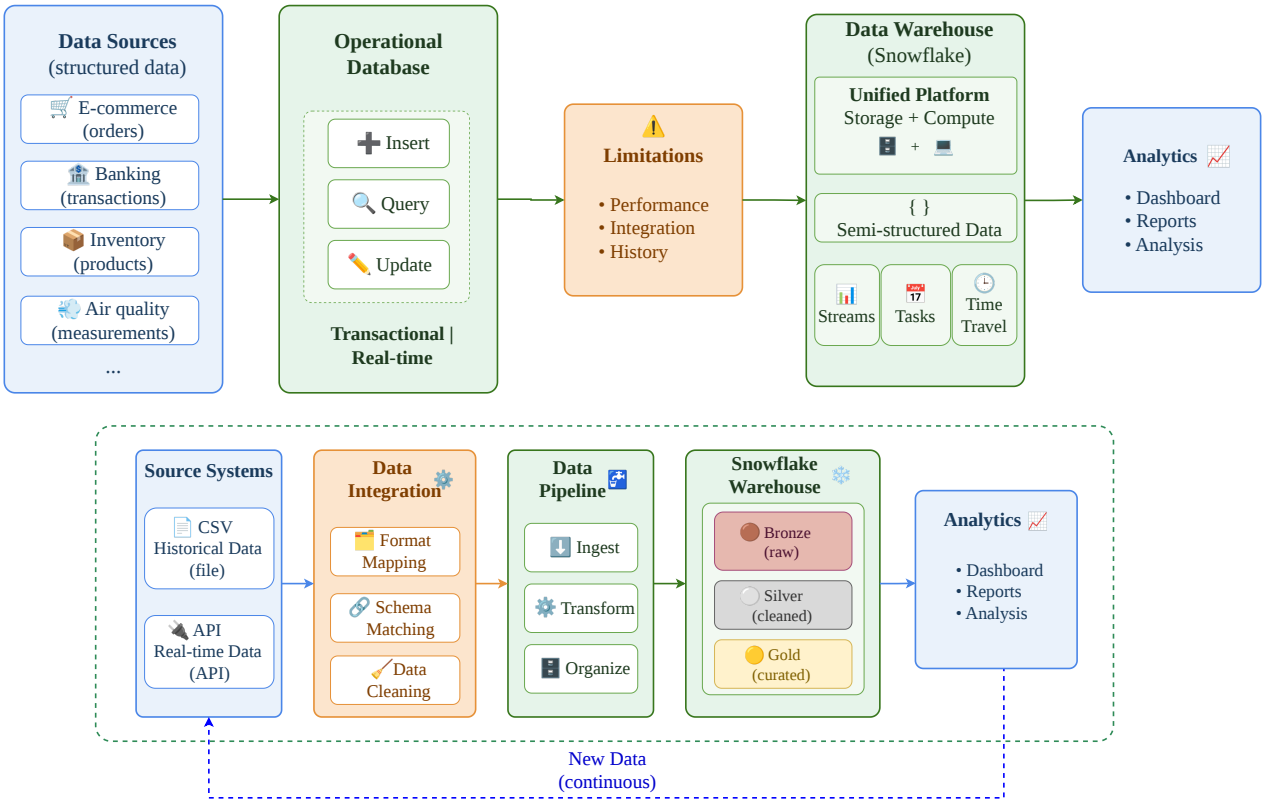
(Data Platform)

Nguyễn Đăng Nhã **Trần Quang Minh** **Đinh Quang Vinh**

Proofreading: Phạm Tiến

I. Giới thiệu

Trong các hệ thống dữ liệu hiện đại, thông tin thường được ghi nhận dưới dạng có cấu trúc, chẳng hạn đơn hàng của hệ thống thương mại điện tử, giao dịch ngân hàng, sản phẩm trong kho hoặc kết quả đo chất lượng không khí tại các trạm quan trắc. Bảng tính có thể đáp ứng nhu cầu ở quy mô nhỏ. Tuy nhiên, khi dữ liệu đạt đến hàng triệu bản ghi và phải được truy xuất thường xuyên theo nhiều điều kiện, hệ thống cần một cơ chế lưu trữ có tổ chức, nhất quán và đáng tin cậy. Cơ chế đó được gọi là **database**.



Hình 1: Dữ liệu từ nhiều nguồn cần được tích hợp, biến đổi trước khi phục vụ các yêu cầu phân tích.

Operational Database trước hết được thiết kế để hỗ trợ hoạt động thường nhật của hệ thống, bao gồm tiếp nhận dữ liệu mới, truy xuất trạng thái hiện tại và cập nhật bản ghi. Khi nhu cầu chuyển từ vận hành sang phân tích, chẳng hạn xác định chất lượng không khí trung bình theo thành phố và theo tháng, kiến trúc này bắt đầu bộc lộ các giới hạn về hiệu năng, khả năng tích hợp và lưu giữ lịch sử. Khi đó, tổ chức cần một hệ thống chuyên biệt cho phân tích được gọi là **data warehouse**.

Snowflake là một **data platform** hiện đại, hỗ trợ xây dựng và vận hành data warehouse. Thay vì kết hợp nhiều dịch vụ độc lập cho lưu trữ, biến đổi và lập lịch, Snowflake tích hợp phần lớn các chức năng này trên một nền tảng thống nhất. Nền tảng sử dụng kiến trúc tách biệt **storage** và **compute**, hỗ trợ **semi-structured data** và cung cấp các cơ chế như `Streams`, `Tasks` và `Time Travel`.

Tuy nhiên, việc có một data warehouse không tự động giải quyết bài toán đưa dữ liệu từ các source system vào hệ thống phân tích. Trong một hệ thống thực tế, dữ liệu có thể đến từ nhiều nguồn với cấu trúc và cách biểu diễn khác nhau. Chẳng hạn, hệ thống theo dõi chất lượng không khí có thể nhận dữ liệu lịch sử từ một file CSV và dữ liệu mới từ một API. Hai nguồn này có thể sử dụng định dạng khác nhau, tên thành phố khác nhau hoặc chứa các trường dữ liệu khác nhau. Nếu dữ liệu được đưa trực tiếp vào cùng một hệ thống mà không có quá trình tích hợp và biến đổi phù hợp, việc xây dựng các truy vấn phân tích sẽ trở nên phức tạp và khó duy trì.

Bên cạnh đó, dữ liệu không chỉ được xử lý một lần. Khi source system tiếp tục phát sinh dữ liệu mới, pipeline cần có khả năng tiếp nhận các bản ghi mới, thực hiện các bước biến đổi cần thiết và cập nhật các lớp dữ liệu phía trên mà không phải xử lý lại toàn bộ dữ liệu. Điều này đặt ra yêu cầu về một **data pipeline** có cấu trúc rõ ràng, có khả năng tự động hóa và duy trì tính nhất quán trong quá trình xử lý.

Trong tài liệu này, bài toán trên được minh họa thông qua dữ liệu chất lượng không khí tại Việt Nam. Dữ liệu được thu thập từ nhiều nguồn và được tổ chức thành các lớp xử lý khác nhau trên Snowflake. Quá trình này cho phép minh họa từ những khái niệm cơ bản của database và data warehouse đến cách xây dựng một data pipeline có khả năng tích hợp, biến đổi và cung cấp dữ liệu cho analytics.

Từ đó, tài liệu lần lượt trình bày các nội dung cần thiết để xây dựng pipeline. Trước hết, phần **Database** trình bày cách dữ liệu được tổ chức và xử lý trong Operational Database, đồng thời phân biệt workload **OLTP** với **OLAP**. Tiếp theo, phần **Data Warehouse** trình bày nhu cầu tách analytical workload khỏi hệ thống vận hành và hai hướng tiếp cận Inmon và Kimball. Phần **Star Schema** trình bày cách chuyển yêu cầu phân tích thành Business Process, Grain, Dimensions và Facts. Sau đó, phần **Snowflake** giới thiệu kiến trúc, các đối tượng cốt lõi và những tính năng tích hợp của nền tảng. Cuối cùng, các khái niệm trên được vận dụng trong dự án **Vietnam Air Quality Data Pipeline** theo kiến trúc `Bronze-Silver-Gold`.

Mục lục

I. Giới thiệu **1**

II. Database cho lưu trữ dữ liệu vận hành **5**

II.1. Database là gì? 5

II.2. Table, row và column 5

II.3. DBMS là gì? 6

II.4. Operational Database và workload OLTP 6

II.5. Dữ liệu vận hành thường được chuẩn hóa 7

III. Từ Database đến Data Warehouse **8**

III.1. Khi nhu cầu chuyển từ vận hành sang Analytics 8

III.2. Vì sao không phân tích trực tiếp trên Operational Database? 9

III.3. Inmon và Kimball: hai hướng xây Data Warehouse 10

III.4. So sánh Inmon và Kimball 12

IV. Mô hình hóa dữ liệu cho Analytics dùng Star Schema **13**

IV.1. Biểu diễn câu hỏi Analytics bằng Measurement và Context 13

IV.2. Xác định Business Process 14

IV.3. Xác định Grain 14

IV.4. Xác định Dimensions 15

IV.5. Xác định Facts 16

IV.6. Ghép Fact và Dimensions thành Star Schema 16

V. Data Platform dùng Snowflake **18**

V.1. Snowflake là gì? 18

V.2. Kiến trúc Snowflake 19

V.3. Core Objects 19

V.4. Stage và Semi-structured Data 20

V.5. Virtual Warehouse 20

V.6. Các tính năng tích hợp quan trọng 21

VI. Project: Vietnam Air Quality Data Pipeline using Snowflake **21**

VI.1. Business Requirement & Multi-source Data Source 21

VI.2. Kiến trúc Medallion 21
VI.3. ELT thay vì ETL 22
VI.4. Lớp CONTROL và bài toán conformance 23
VI.5. Bronze lưu giữ nguyên dữ liệu thô 23
VI.6. Load (bước nạp dữ liệu vào Snowflake) 24
VI.7. Silver được tổ chức theo mô hình Kimball 24
VI.8. Streams, Tasks và cơ chế Orchestration tích hợp 26
VI.9. Data Quality Gate giúp ngăn dữ liệu không đạt yêu cầu 27
VI.10. Gold lưu dữ liệu tổng hợp phục vụ BI 28
VI.11. Lựa chọn Snowflake và kiểm soát chi phí 28

VII. Kết luận 29

VIII. Tài liệu tham khảo 30

Phụ lục 31

II. Database cho lưu trữ dữ liệu vận hành

II.1. Database là gì?

Database

Database là một tập hợp dữ liệu được tổ chức có hệ thống, cho phép quản lý, truy xuất và cập nhật dữ liệu một cách nhất quán.

Database phù hợp với những trường hợp dữ liệu cần được quản lý ổn định, truy cập lặp lại và duy trì tính nhất quán. Bảng tính có thể đáp ứng các tập dữ liệu nhỏ. Tuy nhiên, khi quy mô dữ liệu hoặc số lượng người dùng gia tăng, database cung cấp cơ chế quản trị và truy vấn đáng tin cậy hơn.

II.2. Table, row và column

Table, row và column

Table là bảng dữ liệu, **row** là một dòng, thường biểu diễn một bản ghi, và **column** là một cột, biểu diễn một thuộc tính của bản ghi.

Để diễn giải một bảng dữ liệu, cần phân biệt ba thành phần cơ bản: toàn bộ bảng, từng cột và từng dòng. Ví dụ dưới đây minh họa một bảng lưu kết quả đo chất lượng không khí tại các trạm quan trắc.

Bảng 1: Minh họa các thành phần table, row, column và cell trong một bảng dữ liệu.

station_id	station_name	city	measured_at	aqi
1	HCMC US Consulate	Ho Chi Minh City	2021-01-21 19:00	102
2	**Hanoi US Embassy**	Hanoi	2021-01-21 20:00	281
3	Da Nang	Da Nang	2021-01-21 20:00	120

Khái niệm	Cách nhận biết trong ví dụ
Table	Toàn bộ bảng đo chất lượng không khí, gồm nhiều dòng và nhiều cột.
Column	Cột `station_name`, biểu diễn tên của từng trạm.
Row	Dòng trạm `Hanoi US Embassy`, biểu diễn một bản ghi cụ thể.
Cell	Ô `Hanoi US Embassy` là giao điểm giữa một dòng và một cột.

Lưu ý

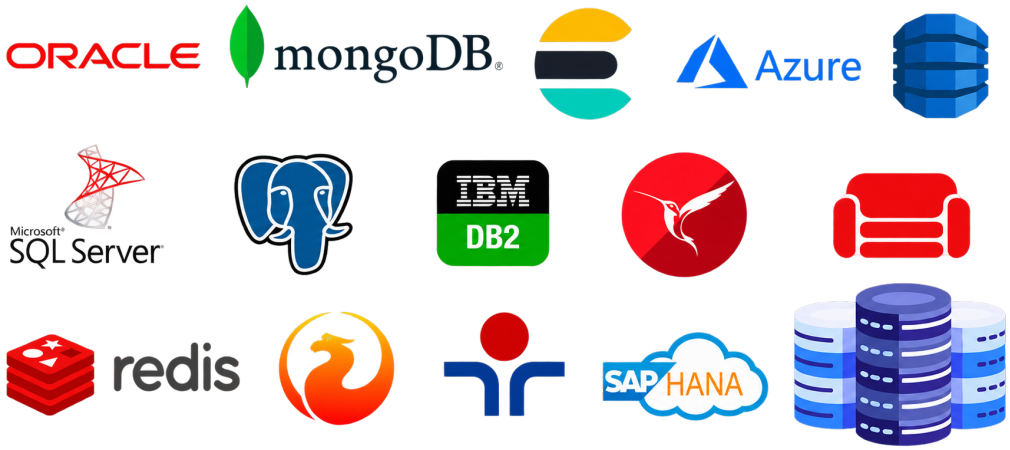
Trong một số tài liệu, **row** còn được gọi là **record** vì mỗi dòng thường biểu diễn một bản ghi cụ thể. Tương tự, **column** có thể được gọi là **field** hoặc **attribute**. Trong data warehouse, các thuộc tính mô tả như thành phố, trạm đo hoặc thời gian thường thuộc **dimension**, trong khi các đại lượng cần phân tích như AQI được xem là **measure** hoặc **fact**.

II.3. DBMS là gì?

DBMS

Database Management System (DBMS) là hệ thống phần mềm dùng để tạo lập, quản lý, truy vấn và cập nhật database. DBMS đóng vai trò trung gian giữa người dùng, câu lệnh truy vấn và dữ liệu được lưu trữ.

Các DBMS khác nhau được thiết kế cho những nhóm yêu cầu khác nhau. Điểm chung của các hệ thống này là tổ chức dữ liệu theo cấu trúc xác định và cung cấp cơ chế truy vấn, cập nhật cũng như kiểm soát truy cập.

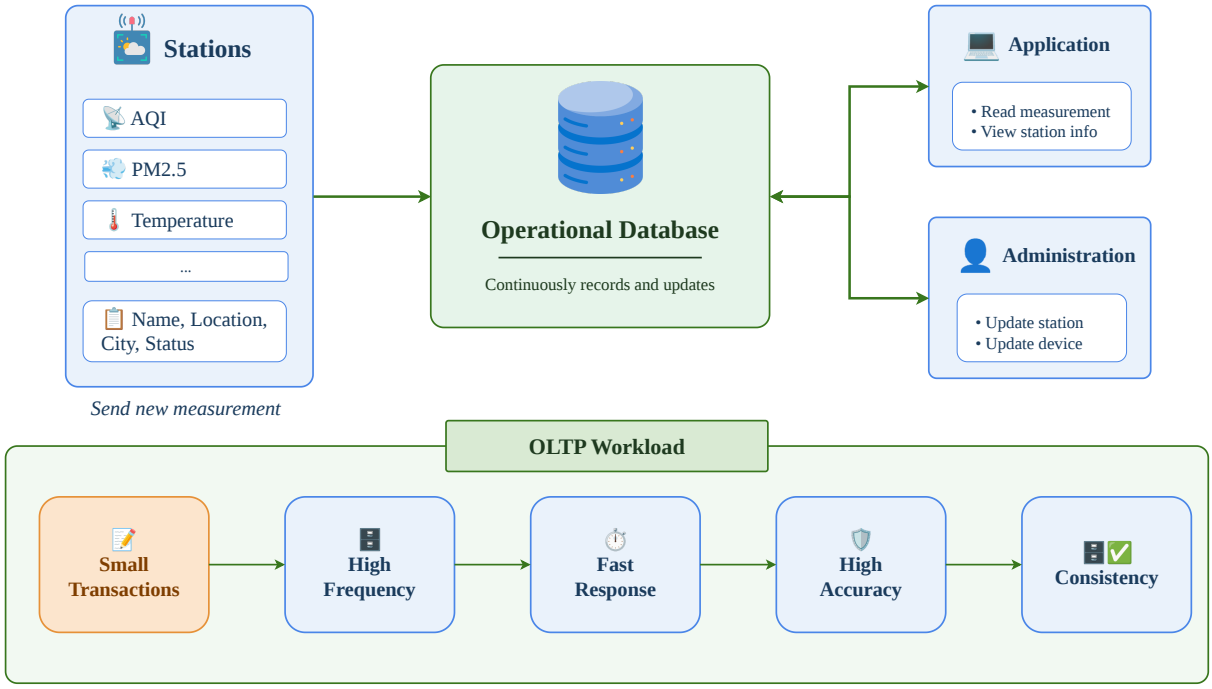


Hình 2: Một số hệ quản trị cơ sở dữ liệu phổ biến (DBMS logos).

II.4. Operational Database và workload OLTP

Trong hệ thống theo dõi chất lượng không khí, nhiều **station** được bố trí tại các thành phố khác nhau và liên tục gửi dữ liệu về hệ thống, bao gồm AQI, PM2.5 và nhiệt độ tại từng thời điểm. Ngoài các đại lượng đo, hệ thống còn lưu thông tin mô tả station như tên trạm, tọa độ, thành phố và trạng thái hoạt động.

Database của hệ thống được thiết kế chủ yếu để hỗ trợ các tác vụ vận hành như tiếp nhận **measurement** mới, truy xuất measurement của một station tại thời điểm cụ thể, cập nhật thông tin station hoặc thay đổi trạng thái thiết bị. Loại database này thường được gọi là **Operational Database** hoặc **Transactional Database**.



Hình 3: Operational Database hỗ trợ các hoạt động thường nhật như tiếp nhận measurement, cung cấp dữ liệu cho application và cập nhật thông tin quản trị

Mỗi thao tác thường chỉ tác động đến một lượng dữ liệu nhỏ, chẳng hạn thêm một measurement, truy xuất trạng thái hiện tại của station hoặc cập nhật thông tin thiết bị. Tuy nhiên, các thao tác diễn ra với tần suất cao và đòi hỏi thời gian phản hồi ngắn, độ chính xác cao và tính nhất quán giao dịch.

Kiểu **workload** này được gọi là **Online Transaction Processing (OLTP)**. OLTP tập trung xử lý số lượng lớn các giao dịch có phạm vi nhỏ, chẳng hạn thêm, sửa, xóa hoặc truy xuất một số ít bản ghi trong quá trình hệ thống hoạt động.

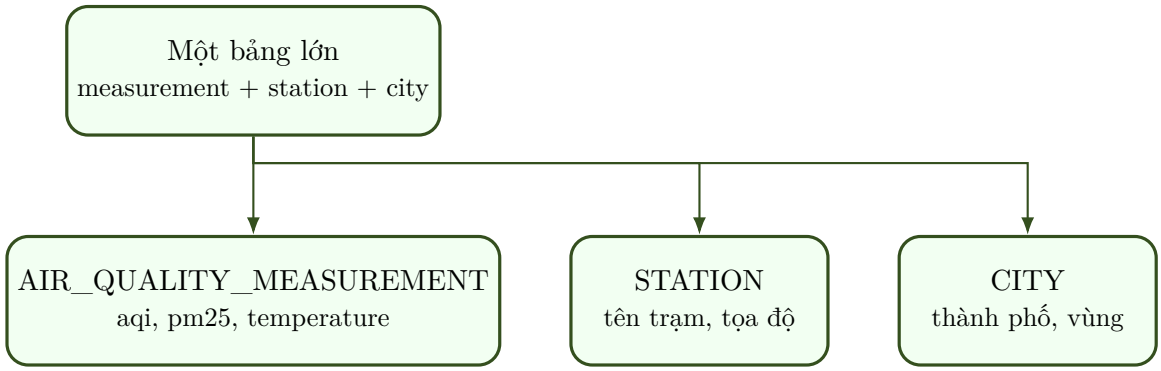
II.5. Dữ liệu vận hành thường được chuẩn hóa

Giả sử measurement và thông tin station ban đầu được lưu trong cùng một bảng:

station_id	station_name	lat	lon	city	measured_at	aqi	pm25	temperature
8767	HCMC US Consulate	10.782978	106.700711	Ho Chi Minh City	2021-01-21 19:00	102	102	27.4
8767	HCMC US Consulate	10.782978	106.700711	Ho Chi Minh City	2021-01-21 20:00	108	108	27.1
8641	Hanoi US Embassy	21.021506	105.818875	Hanoi	2021-01-21 20:00	281	281	18.5

Cách tổ chức này trực quan nhưng làm lặp lại thông tin station trong mọi measurement. Nếu station 8767 gửi hàng trăm measurement mỗi ngày, tên trạm, tọa độ và thành phố cũng được lưu lại với tần suất tương ứng. Hệ quả không chỉ là gia tăng dung lượng lưu trữ mà khi thông tin station thay đổi, hệ thống còn phải bảo đảm tất cả bản ghi liên quan được cập nhật nhất quán.

Do đó, Operational Database thường áp dụng **normalization** nhằm tách các nhóm thông tin có vai trò khác nhau thành những bảng riêng biệt.



Hình 4: Normalization phân rã một bảng lớn thành các bảng có chức năng riêng nhằm giảm dữ liệu lặp và duy trì tính nhất quán khi cập nhật.

Sau khi chuẩn hóa, dữ liệu measurement được tách khỏi dữ liệu station. Các dạng chuẩn như `1NF`, `2NF` và `3NF` mô tả những mức độ normalization khác nhau. Trong phạm vi tài liệu này, trọng tâm không nằm ở việc trình bày chi tiết từng **normal form**, mà ở nguyên tắc tổ chức dữ liệu sau:

Tư tưởng của Operational Database

Operational Database ưu tiên cấu trúc dữ liệu hỗ trợ hiệu quả các thao tác **ghi** và **cập nhật**, đồng thời duy trì tính nhất quán.

III. Từ Database đến Data Warehouse

III.1. Khi nhu cầu chuyển từ vận hành sang Analytics

Theo thời gian, dữ liệu chất lượng không khí không chỉ phục vụ vận hành ứng dụng mà còn trở thành cơ sở cho các yêu cầu phân tích. Những yêu cầu điển hình bao gồm xác định AQI trung bình theo thành phố và theo tháng, nhận diện thành phố có PM2.5 cao nhất trong một năm, so sánh chất lượng không khí giữa các mùa tại Thành phố Hồ Chí Minh, hoặc xác định station thường xuyên ghi nhận AQI ở mức cao.

Đây là một loại workload khác với workload vận hành. Thay vì chỉ truy xuất một measurement hoặc cập nhật một station, các yêu cầu trên đòi hỏi xử lý dữ liệu trên phạm vi rộng, bao gồm nhiều station, nhiều khu vực và khoảng thời gian kéo dài nhiều tháng hoặc nhiều năm, sau đó thực hiện các phép lọc, nhóm, tổng hợp và so sánh.

Kiểu workload này được gọi là **Online Analytical Processing (OLAP)**. OLAP tập trung vào việc đọc và xử lý lượng dữ liệu lớn nhằm hỗ trợ các phép lọc, nhóm, tổng hợp và so sánh đa chiều.

Một khác biệt quan trọng là hoạt động **analytics** không phải lúc nào cũng yêu cầu dữ liệu được cập nhật ngay sau mỗi thay đổi. Tổ chức có thể lập lịch cho data pipeline xử lý theo **batch** vào ban đêm, thu thập dữ liệu từ các **source system** và chuẩn bị tập dữ liệu phân tích. Sau đó, **data analyst** có thể sử dụng dữ liệu này để cập nhật **dashboard** hoặc xây dựng **report**.

Bảng 2: So sánh các đặc trưng cơ bản của workload OLTP và OLAP.

	OLTP	**OLAP**
Mục đích chính	hỗ trợ hoạt động thường nhật	hỗ trợ phân tích dữ liệu
Thao tác phổ biến	thêm, sửa, xóa, truy xuất ít bản ghi	đọc, lọc, nhóm, tổng hợp, so sánh
Phạm vi mỗi query	thường nhỏ	thường lớn
Dữ liệu trọng tâm	trạng thái hiện tại	dữ liệu tích hợp, thường bao gồm lịch sử

III.2. Vì sao không phân tích trực tiếp trên Operational Database?

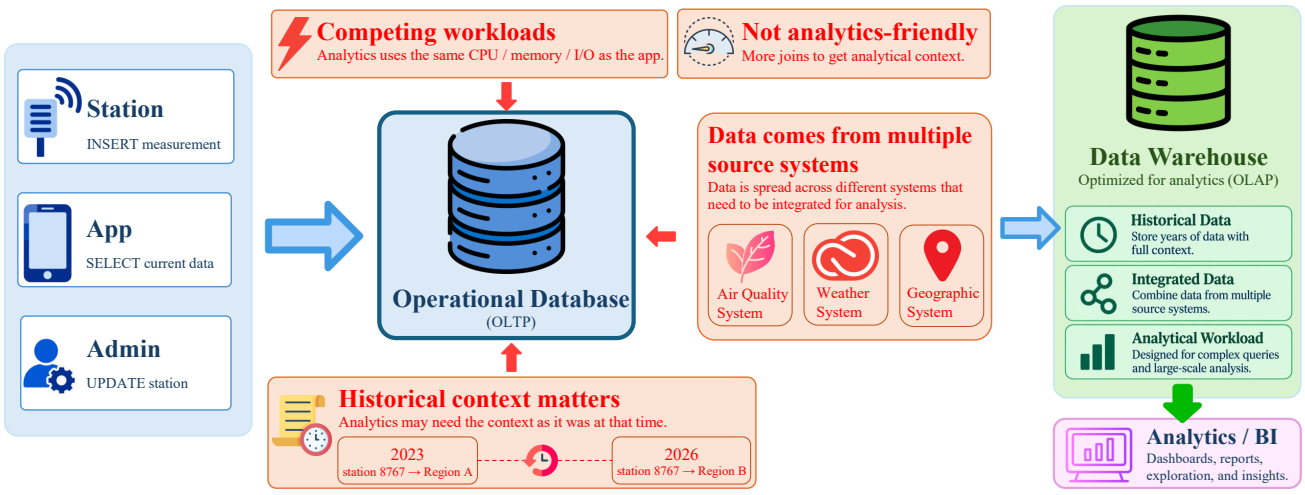
Về mặt kỹ thuật, dữ liệu có thể được truy vấn trực tiếp trên Operational Database để thực hiện analytics, và phương án này có thể đáp ứng một hệ thống quy mô nhỏ. Tuy nhiên, khi khối lượng dữ liệu, số lượng người dùng phân tích và độ phức tạp của truy vấn gia tăng, bốn nhóm vấn đề chính thường xuất hiện.

Thứ nhất, operational workload và analytical workload có đặc trưng khác nhau. Operational Database phải liên tục tiếp nhận measurement mới và cung cấp dữ liệu hiện tại cho ứng dụng. Trong khi đó, một analyst có thể thực thi truy vấn tính AQI trung bình theo station, thành phố và tháng trong năm năm, qua đó quét hàng triệu dòng dữ liệu. Khi hai loại workload dùng chung CPU, bộ nhớ và thông lượng nhập/xuất, analytical workload có thể cạnh tranh tài nguyên với các giao dịch thiết yếu của ứng dụng.

Thứ hai, cấu trúc dữ liệu phục vụ vận hành không nhất thiết tối ưu cho analytics. Operational Database thường tổ chức dữ liệu theo **normalized model**. Cấu trúc này phù hợp với operational workload, nhưng một analytical query có thể phải kết nối nhiều bảng để thu thập đầy đủ context. Chẳng hạn, đối với yêu cầu tính AQI trung bình theo thành phố và theo tháng, AQI nằm trong `AIR_QUALITY_MEASUREMENT`, còn thông tin thành phố phải được truy xuất thông qua `STATION` và `CITY`. Do đó, cấu trúc tối ưu cho ghi và cập nhật chưa chắc tối ưu cho phân tích.

Thứ ba, dữ liệu phân tích thường được tích hợp từ nhiều source system. Một bài toán analytics thực tế hiếm khi chỉ sử dụng một database. Ví dụ, Air Quality System lưu AQI và thông tin station, Weather System lưu nhiệt độ và độ ẩm, Geographic System quản lý thành phố và vùng địa lý. Trước khi phân tích, dữ liệu từ các nguồn này phải được tích hợp và chuẩn hóa về cùng một hệ quy chiếu.

Thứ tư, analytics thường yêu cầu historical context. Ứng dụng vận hành chủ yếu quan tâm đến trạng thái hiện tại, trong khi phân tích có thể cần xác định station thuộc vùng nào tại thời điểm một measurement được ghi nhận. Nếu thông tin cũ bị ghi đè khi trạng thái thay đổi, bối cảnh lịch sử cần thiết cho phân tích sẽ không còn được bảo toàn.



Hình 5: Tổng hợp bốn vấn đề khi đặt analytical workload trực tiếp lên Operational Database, dẫn đến nhu cầu tách một analytical platform riêng.

Khi các yêu cầu trên gia tăng, giải pháp phù hợp là **tách analytical workload khỏi hệ thống vận hành**.

Data Warehouse

Data Warehouse là hệ thống được xây dựng chủ yếu để tích hợp dữ liệu từ nhiều source system, lưu giữ dữ liệu phục vụ phân tích và hỗ trợ analytical workload trên quy mô lớn.

Vì vậy, việc một tổ chức đã vận hành nhiều database không loại trừ nhu cầu xây dựng một data warehouse chuyên biệt.

III.3. Inmon và Kimball: hai hướng xây Data Warehouse

Sau khi xác lập nhu cầu đối với data warehouse, vấn đề tiếp theo là cách tổ chức dữ liệu đã được thu thập và tích hợp từ nhiều source system. Hai hướng tiếp cận có ảnh hưởng lớn trong lĩnh vực này là **Inmon** và **Kimball**.

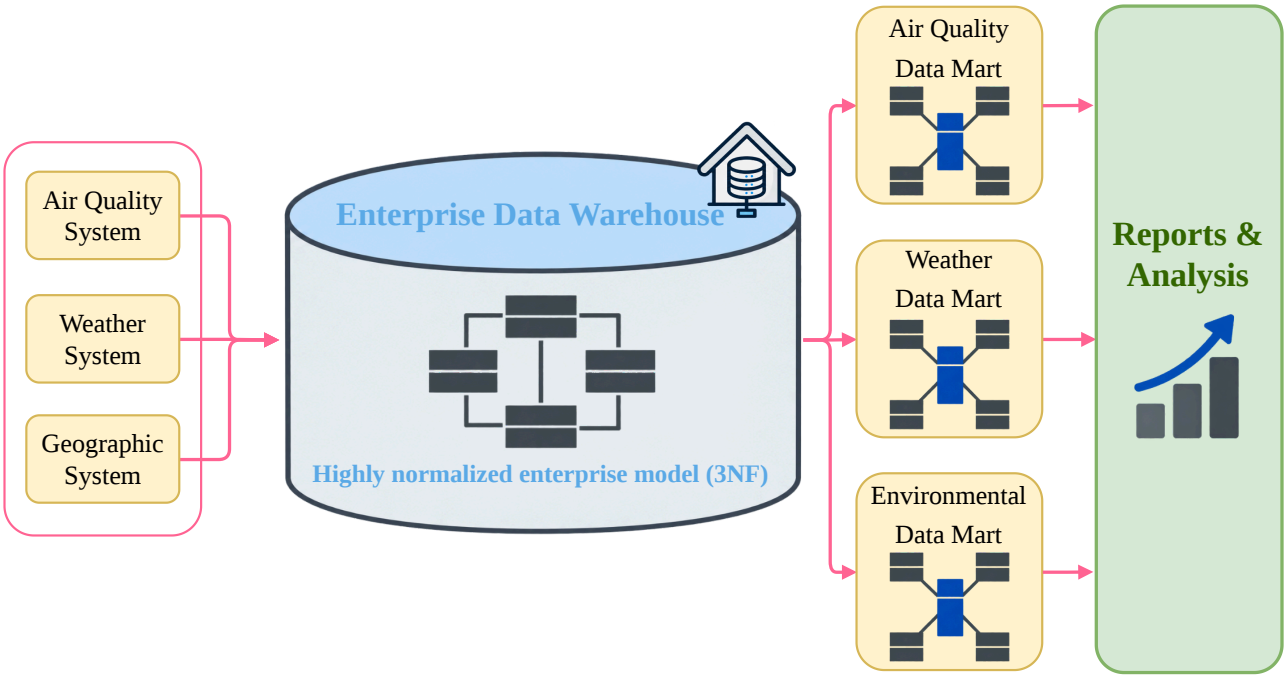
Inmon bắt đầu bằng việc xây dựng nền dữ liệu tích hợp cho toàn doanh nghiệp, sau đó tạo các tập dữ liệu phục vụ từng nhu cầu cụ thể. Ngược lại, **Kimball** khởi đầu từ các business process có yêu cầu phân tích rõ ràng và mở rộng dần thành một data warehouse có tính tích hợp.

III.3.1. Hướng tiếp cận của Inmon

Bill Inmon định nghĩa Data Warehouse là một tập dữ liệu có bốn đặc trưng: **subject-oriented**, **integrated**, **time-variant** và **non-volatile**, được xây dựng nhằm hỗ trợ quá trình ra quyết định. Trọng tâm của cách tiếp cận này là xây dựng trước một nền dữ liệu chung trên phạm vi toàn doanh nghiệp.

Thay vì để từng nhóm analytics tự thu thập và tổ chức dữ liệu theo cách riêng, phương pháp Inmon tích hợp các nguồn vào một **Enterprise Data Warehouse (EDW)** dùng chung, thường được tổ chức dưới dạng **highly normalized model**. Khi một nhóm người dùng cần cấu trúc

dữ liệu thuận tiện hơn cho một phạm vi phân tích cụ thể, dữ liệu liên quan được trích xuất từ EDW để xây dựng **Data Mart** chuyên biệt.



Hình 6: Inmon Architecture. Dữ liệu được tích hợp vào Enterprise Data Warehouse trước khi phân phối xuống các Data Mart phục vụ phân tích.

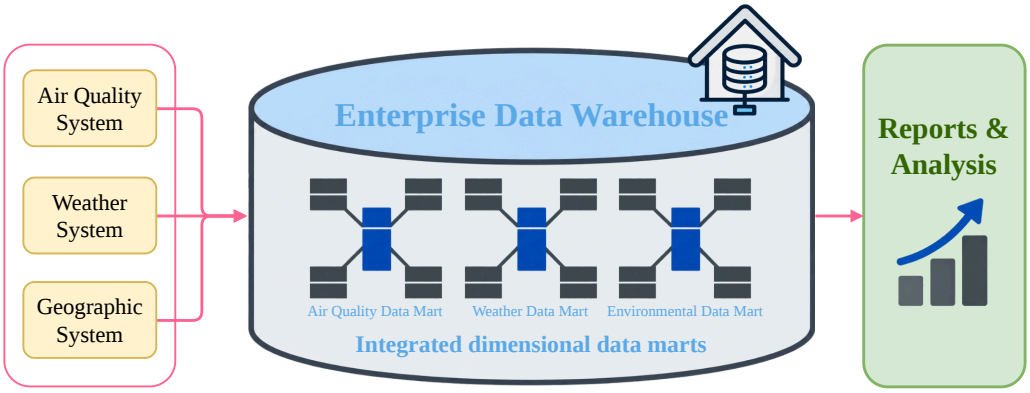
Tóm lại, tư tưởng Inmon có thể được mô tả bằng nguyên tắc **enterprise-first**: xây dựng nền dữ liệu thống nhất cho toàn doanh nghiệp trước, sau đó phát triển các thành phần phục vụ từng phạm vi phân tích. Cách tiếp cận này thường được gọi là **top-down approach**.

III.3.2. Hướng tiếp cận của Kimball

Ralph Kimball đề xuất một trình tự khác. Thay vì yêu cầu hoàn thành trước một EDW đã chuẩn hóa, Kimball bắt đầu từ các **business process** có nhu cầu phân tích rõ ràng. Trong ví dụ về chất lượng không khí, một business process có thể là **Air Quality Measurement**: mỗi lần station thực hiện phép đo tại một thời điểm sẽ tạo ra các giá trị AQI, PM2.5 và nhiệt độ.

Theo phương pháp Kimball, tổ chức có thể xây dựng trước tập dữ liệu phục vụ business process này mà không cần chờ hoàn thiện **enterprise model**. Khi xuất hiện nhu cầu phân tích nhiệt độ và độ ẩm, data warehouse có thể được mở rộng với business process **Weather Measurement**. Như vậy, Data Warehouse được phát triển theo phương thức **incremental**.

Tuy nhiên, phương thức incremental không đồng nghĩa với việc xây dựng các Data Mart độc lập. Trong kiến trúc Kimball, các Data Mart phải được thiết kế để có thể tích hợp thông qua những định nghĩa dùng chung. Các khái niệm như station, location và time do đó cần được chuẩn hóa nhất quán giữa các business process.



Hình 7: Lưu trữ dữ liệu theo kiến trúc Kimball. Dữ liệu từ nhiều hệ thống nguồn được tổ chức thành các dimensional Data Mart theo từng business process và tích hợp với nhau để phục vụ báo cáo và phân tích.

Tư tưởng Kimball có thể được mô tả bằng nguyên tắc **business-process-first**: bắt đầu từ một business process có giá trị phân tích, triển khai phạm vi đó trước và mở rộng dần trong khi vẫn duy trì khả năng tích hợp. Cách tiếp cận này thường được gọi là **bottom-up approach**.

III.4. So sánh Inmon và Kimball

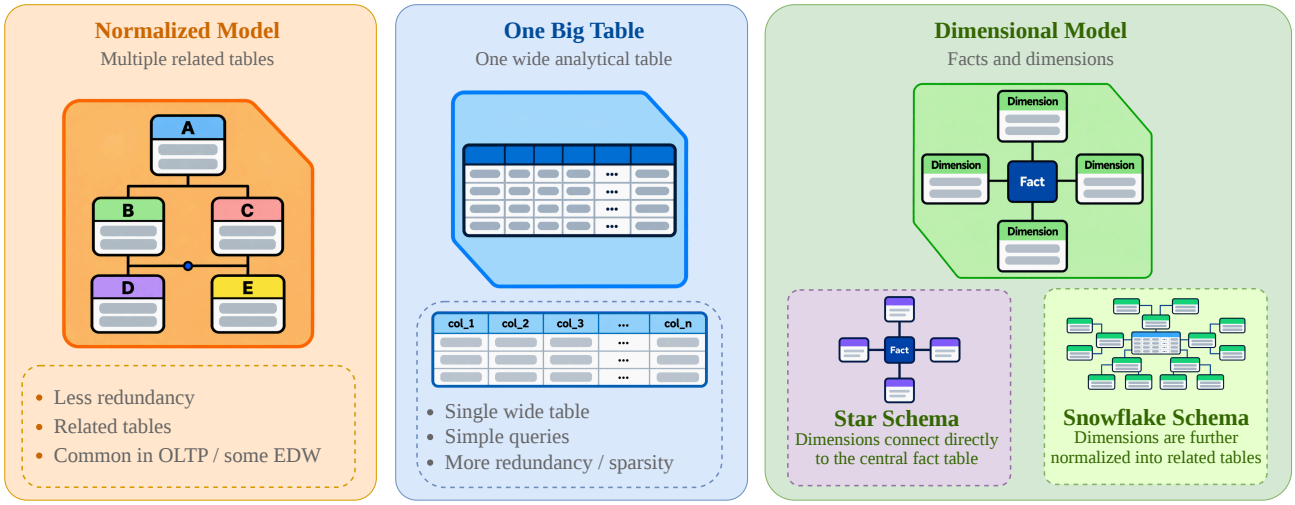
Cả Inmon và Kimball đều yêu cầu thu thập, làm sạch và tích hợp dữ liệu từ nhiều source system. Khác biệt chủ yếu nằm ở **điểm khởi đầu và phương thức phát triển Data Warehouse theo thời gian**.

	Inmon	**Kimball**
Cách bắt đầu	xây nền dữ liệu chung cho toàn doanh nghiệp	bắt đầu từ từng business process có nhu cầu phân tích
Hướng phát triển	top-down	bottom-up
Nền dữ liệu ban đầu	Enterprise Data Warehouse, thường normalized	Data Mart theo từng business process
Data Mart	được xây từ EDW	được xây từng phần nhưng phải tích hợp với nhau
Ưu điểm	tính thống nhất cấp doanh nghiệp được thiết kế từ đầu	tạo giá trị từng phần mà không cần chờ hoàn thiện toàn bộ
Hạn chế	yêu cầu modeling và nguồn lực lớn ở giai đoạn đầu	yêu cầu governance hiệu quả để duy trì tính thống nhất giữa các Data Mart

Inmon và Kimball giải quyết vấn đề ở cấp độ **architecture**: Data Warehouse nên được xây dựng và mở rộng theo phương thức nào. Ở cấp độ **data model**, vấn đề còn lại là cách tổ chức các bảng để hỗ trợ phân tích hiệu quả. Nội dung này được trình bày trong phần tiếp theo.

IV. Mô hình hóa dữ liệu cho Analytics dùng Star Schema

Như đã phân tích, analytical workload không chỉ giới hạn ở việc truy xuất một số bản ghi cụ thể, mà thường yêu cầu đọc lượng dữ liệu lớn để lọc, nhóm, tổng hợp và so sánh theo nhiều góc nhìn. Vì vậy, dữ liệu cần được tổ chức phù hợp với cách người dùng hình thành câu hỏi phân tích. Đây là cơ sở của **Dimensional Modeling**.



Hình 8: Ba hướng tổ chức analytical data phổ biến, gồm Normalized Model, One Big Table và Dimensional Model; trong đó Dimensional Model gồm Star Schema và Snowflake Schema.

Trong phạm vi tài liệu này, trọng tâm là **Star Schema**, một cấu trúc phổ biến của Dimensional Modeling. Quá trình thiết kế không bắt đầu từ các bảng nguồn hiện có, mà từ yêu cầu và câu hỏi analytics cần được hỗ trợ.

IV.1. Biểu diễn câu hỏi Analytics bằng Measurement và Context

Xét câu hỏi phân tích: *AQI trung bình theo thành phố trong năm 2021 là bao nhiêu?* Ở bước đầu, vị trí vật lý của dữ liệu trong các bảng nguồn chưa phải là yếu tố trọng tâm.

Câu hỏi có thể được phân rã thành hai thành phần. Thành phần thứ nhất là **measurement**, trong trường hợp này là `AQI` với phép tổng hợp `AVG(aqi)`. Thành phần thứ hai là **context**: `city` xác định chiều phân nhóm theo thành phố, còn điều kiện `year = 2021` giới hạn phạm vi thời gian.

Thành phần	Giá trị
Measurement	AQI
Group by	City
Filter	Year = 2021

Người dùng thực hiện analytics thường bắt đầu từ **đại lượng cần đo** và **góc nhìn dùng để phân tích đại lượng đó**, thay vì từ cấu trúc liên kết giữa các bảng. Tuy nhiên, trước khi tổ chức measurement và context, cần xác định sự kiện nghiệp vụ đã tạo ra đại lượng cần phân tích.

IV.2. Xác định Business Process

Measurement được hình thành từ một hoạt động hoặc sự kiện nghiệp vụ cụ thể. Trong dữ liệu chất lượng không khí, AQI, PM2.5 và nhiệt độ được ghi nhận mỗi khi station thực hiện phép đo tại một thời điểm xác định. Mỗi lần ghi nhận như vậy được xem là một **event**.

Business Process

Business Process là hoạt động hoặc quá trình tạo ra các event cần được đo lường và phân tích.

Trong ví dụ hiện tại, Business Process được xác định là **Air Quality Measurement**. Việc xác định Business Process giúp giới hạn rõ phạm vi của data model. Mô hình không được xây dựng riêng quanh `STATION`, `CITY` hoặc `REGION`, mà quanh các lần Air Quality Measurement được ghi nhận.

Câu hỏi định hướng cho bước này là **What happened?** Trong ví dụ, câu trả lời là một Air Quality Measurement đã được ghi nhận. Tuy nhiên, cùng một Business Process có thể được lưu ở nhiều mức độ chi tiết. Vì vậy, bước tiếp theo là xác định **grain**.

IV.3. Xác định Grain

Giả sử station `8767` ghi nhận ba measurement trong một ngày. Khi đưa dữ liệu vào Data Warehouse, có ít nhất hai phương án: bảo toàn từng measurement hoặc tổng hợp trước thành một row đại diện cho cả ngày.

station_id	measured_at	aqi
8767	2021-01-21 19:00	102
8767	2021-01-21 20:00	108
8767	2021-01-21 21:00	111

station_id	date	avg_aqi
8767	2021-01-21	107

Cả hai bảng đều mô tả cùng một Business Process, nhưng khác nhau về **mức độ chi tiết được biểu diễn bởi mỗi row**.

Grain

Grain xác định mức độ chi tiết của dữ liệu, cụ thể là ý nghĩa của một row trong tập dữ liệu.

Câu hỏi định hướng khi xác định grain là **What does one row represent?**. Grain quyết định phạm vi câu hỏi mà warehouse có thể trả lời về sau. Nếu chỉ lưu `avg_aqi` theo ngày, hệ thống vẫn có thể phân tích theo ngày, tháng hoặc năm, nhưng không thể khôi phục AQI tại 19:00 hoặc 20:00. Vì vậy, Dimensional Modeling thường ưu tiên **atomic grain**, tức mức chi tiết thấp nhất còn có ý nghĩa nghiệp vụ. Trong ví dụ này, grain được xác định như sau: *mỗi row biểu diễn một Air Quality Measurement tại một station và một thời điểm cụ thể.*

IV.4. Xác định Dimensions

Một measurement có thể cho biết `AQI = 102`, nhưng giá trị riêng lẻ này chưa đủ để hỗ trợ phân tích. Cần bổ sung thông tin về station ghi nhận, vị trí của station, ngày đo và thời điểm đo. Các thuộc tính này cấu thành context của business event.

Dimension

Dimension chứa các thuộc tính mô tả context của một business event, qua đó cho phép người dùng phân tích dữ liệu theo nhiều góc nhìn.

Trong analytics, thuộc tính của Dimension thường được sử dụng để lọc, phân nhóm hoặc hiển thị kết quả. Các thuộc tính này thường trả lời nhóm câu hỏi **5W1H**:

Câu hỏi	Context thường gặp
Who?	ai hoặc đối tượng nào tham gia
What?	sản phẩm gì, station nào, loại nào
Where?	ở đâu
When?	khi nào
Why?	vì sao
How?	theo cách nào, qua kênh nào

Đối với Air Quality Measurement, ba nhóm context quan trọng gồm station thực hiện phép đo, vị trí của station theo city/province/region, và ngày giờ ghi nhận measurement. Từ đó, context được tổ chức thành các Dimension chính như `DIM_STATION`, `DIM_DATE` và `DIM_TIME`.

Ví dụ, `DIM_STATION` mô tả station gắn với mỗi measurement:

station_key	station_id	station_name	city	province	region
1	8767	HCMC US Consulate	Ho Chi Minh City	Ho Chi Minh City	South
2	8641	Hanoi US Embassy	Hanoi	Hanoi	North

Thiết kế này thể hiện một khác biệt quan trọng so với normalized model. Trong Operational Database, `STATION`, `CITY`, `PROVINCE` và `REGION` có thể được tách thành nhiều bảng nhằm hạn chế dữ liệu lặp. Trong Dimensional Modeling, các thuộc tính cùng mô tả một station được đặt trong `DIM_STATION`. Một mức độ **denormalization** được chấp nhận để đơn giản hóa analytical query.

Thời gian là một Dimension đặc biệt quan trọng. Analyst thường phân tích cùng một measurement theo nhiều cấp độ thời gian như ngày, tuần, tháng, quý, năm hoặc khung giờ trong ngày. Các cấp độ này có thể được biểu diễn sẵn trong `DIM_DATE` và `DIM_TIME`:

date_key	date	day	month	quarter	year
20210121	2021-01-21	21	1	1	2021

IV.5. Xác định Facts

Sau khi xác định Business Process, grain và các Dimension, bước tiếp theo là nhận diện các đại lượng định lượng trong một measurement:

station_id	measured_at	aqi	pm25	temperature
8767	2021-01-21 19:00	102	102	27.4

Trong mỗi phép đo, `aqi`, `pm25` và `temperature` là các giá trị được ghi nhận trực tiếp, đồng thời là các đại lượng người dùng cần tính toán, tổng hợp hoặc so sánh.

Fact

Fact là các measurement được ghi nhận tại grain đã xác định và được sử dụng trong các phép tính, tổng hợp hoặc so sánh.

Nếu Dimension thường trả lời các câu hỏi **Who, What, Where, When, Why, How**, thì Fact trả lời các câu hỏi định lượng như **How much? How many? How long?**. Đối với Air Quality Measurement, các Fact gồm `aqi`, `pm25` và `temperature`.

Cần lưu ý rằng **Fact không đồng nghĩa với mọi cột có kiểu số**. Giá trị `station_id = 8767` cũng có dạng số, nhưng phép tính tổng trên `station_id` không mang ý nghĩa nghiệp vụ; thuộc tính này được dùng để nhận diện station và cung cấp context. Do đó, một Fact phải là measurement tại grain của Business Process và có ý nghĩa đối với phép tính phân tích.

IV.6. Ghép Fact và Dimensions thành Star Schema

Sau khi xác định Business Process, grain, Dimensions và Facts, các thành phần này được tổ chức thành các bảng có quan hệ với nhau.

Star Schema

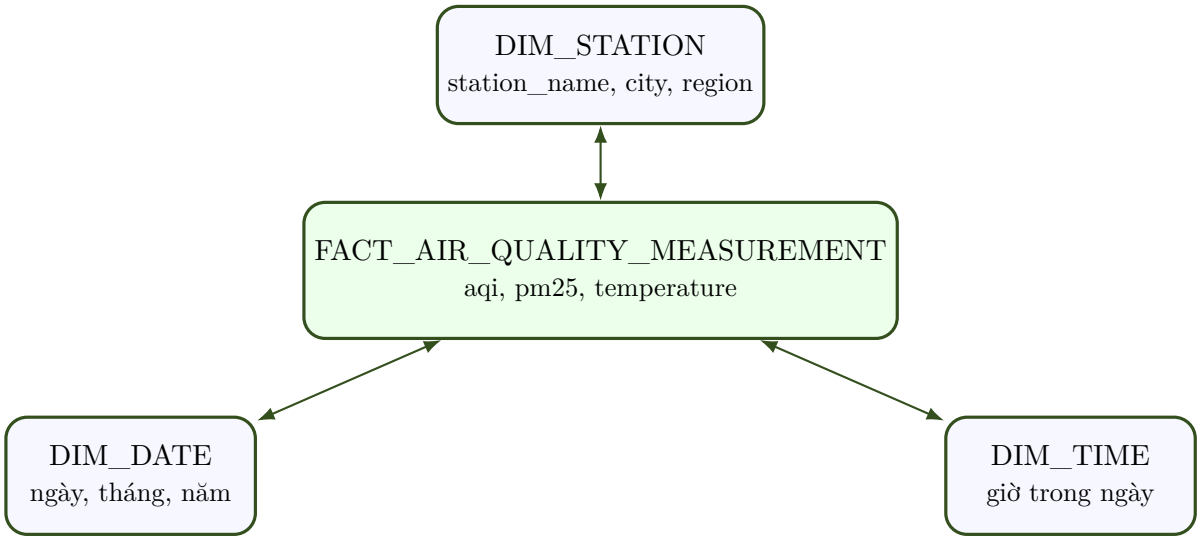
Star Schema là một Dimensional Model trong đó Fact Table nằm ở trung tâm và liên kết trực tiếp với các Dimension Table xung quanh.

Quy trình thiết kế có thể được khái quát như sau:

Trong Star Schema, Fact Table nằm ở trung tâm và lưu các Fact của Business Process tại grain đã xác định. Các Dimension Table bao quanh Fact Table, mỗi Dimension cung cấp một nhóm context để mô tả, lọc và phân nhóm Fact.

Bảng 3: Bốn bước chuyển từ câu hỏi analytics sang các thành phần của Star Schema.

Bước	**Câu hỏi cần trả lời**	**Air Quality**
Business Process	What happened?	Một Air Quality Measurement được ghi nhận
Grain	Một row đại diện cho điều gì?	Một measurement tại một station ở một thời điểm
Dimensions	Who / What / Where / When / Why / How?	Station, location, date, time
Facts	How much / How many / How long?	AQI, PM2.5, temperature



Hình 9: Star Schema của Air Quality Measurement. Fact Table ở trung tâm và các Dimension Table cung cấp context xung quanh.

Fact Table có thể được tổ chức như sau:

station_key	**date_key**	**time_key**	**aqi**	**pm25**	**temperature**
1	20210121	1900	102	102	27.4
1	20210121	2000	108	108	27.1
1	20210121	2100	111	111	26.8

Mỗi Dimension thường có một key riêng để Fact Table tham chiếu. Ví dụ, `station_id = 8767` là **identifier** đến từ source system, trong khi `station_key = 1` có thể là **surrogate key** do Data Warehouse tạo ra. Như vậy, `station_id` mang định danh nghiệp vụ hoặc định danh nguồn, còn `station_key` là định danh nội bộ được warehouse sử dụng để liên kết dữ liệu.

Star Schema không được hình thành từ việc phân chia bảng một cách tùy ý. Mỗi thành phần của mô hình tương ứng với một câu hỏi thiết kế cụ thể:

Analytics Question → Measurement + Context → Business Process → Grain → Dimensions → Facts → Star Schema

Đối với câu hỏi *AQI trung bình theo thành phố trong năm 2021 là bao nhiêu?*, cấu trúc của model phản ánh trực tiếp cấu trúc của yêu cầu phân tích: `AVG(aqi)` được tính từ Fact, `city` nằm trong `DIM_STATION`, còn `year = 2021` nằm trong `DIM_DATE`. Giá trị cốt lõi của Star Schema là tổ chức dữ liệu quanh **Business Process**, đặt các **measurement cần phân tích ở trung tâm** và liên kết chúng trực tiếp với những **context thường được analyst sử dụng**.

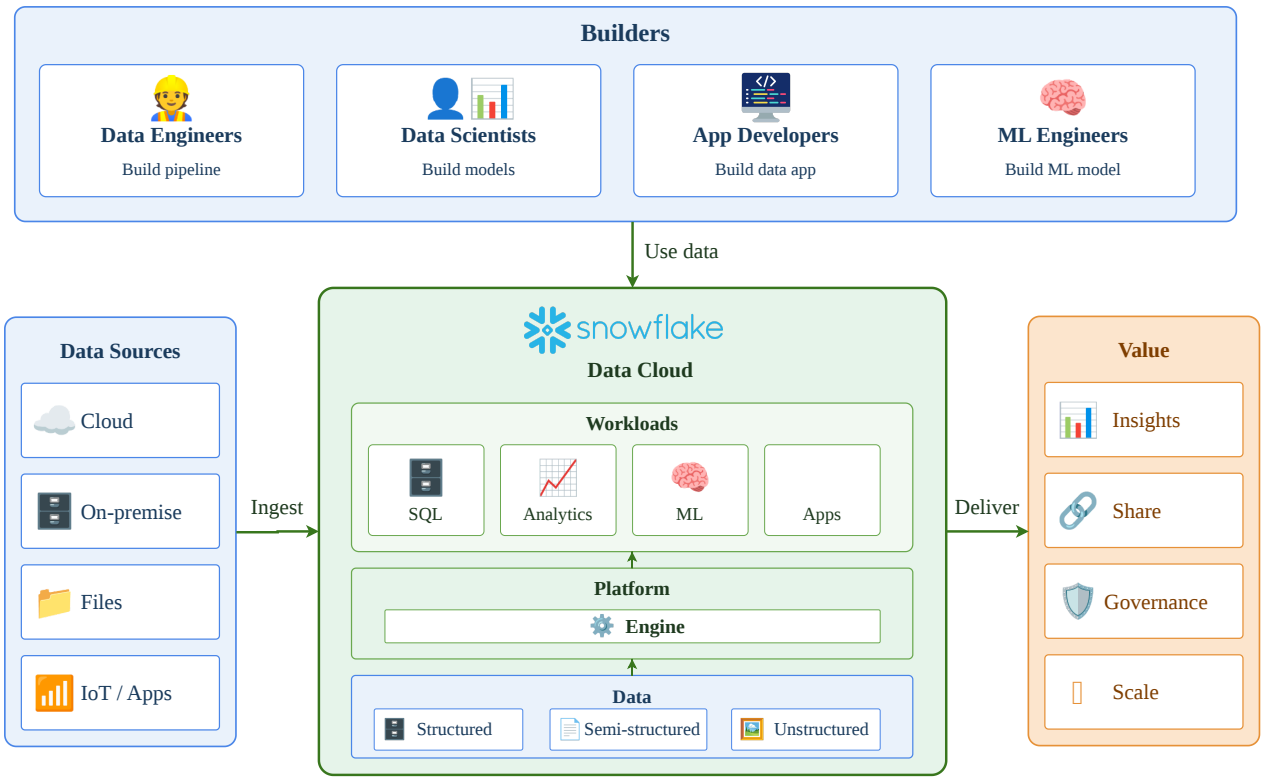
V. Data Platform dùng Snowflake

V.1. Snowflake là gì?

Snowflake Data Cloud

Snowflake là một **Data Platform**, được phát triển theo định hướng **Data Cloud**, cho phép tổ chức xử lý dữ liệu ở quy mô lớn, chia sẻ dữ liệu có kiểm soát và vận hành nhiều workload trên một nền tảng thống nhất.

Theo mô hình khái niệm của Snowflake, Data Cloud bao gồm hai thành phần chính là **platform** và **data**, trong đó platform cung cấp **engine** cho các năng lực của Snowflake. Mối quan hệ giữa dữ liệu, nền tảng và workload được minh họa như sau:

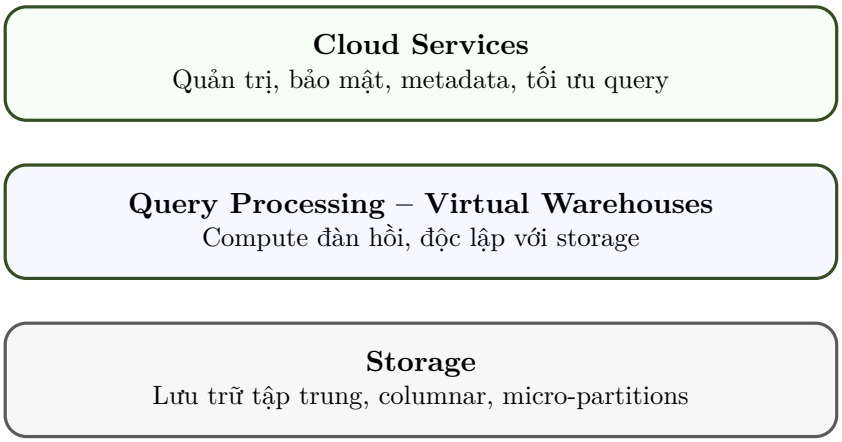


Hình 10: Mô hình Data Cloud của Snowflake. Platform xử lý dữ liệu để phục vụ nhiều loại workload.

Snowflake hỗ trợ nhiều nhóm chuyên môn, được gọi chung là **Builders**. Các nhóm này gồm **Data Engineers**, **Data Scientists**, **Data Application Developers** và **Machine Learning Engineers**, họ sử dụng dữ liệu để xây dựng pipeline, mô hình thống kê, **machine learning model** hoặc **data application**. Do đó, Snowflake không chỉ là môi trường thực thi SQL mà còn là một platform phục vụ nhiều vai trò và workload dữ liệu.

V.2. Kiến trúc Snowflake

Đặc trưng kiến trúc quan trọng của Snowflake là **sự tách biệt giữa storage và compute**. Hệ thống gồm ba **layer**:



Hình 11: Ba layer trong kiến trúc Snowflake gồm Cloud Services, Virtual Warehouses (compute) và Storage.

Ba layer đảm nhiệm các chức năng khác nhau. **Cloud Services** quản lý **authentication**, metadata, bảo mật, transaction và **query optimization**. **Virtual Warehouses** cung cấp các cụm compute có khả năng mở rộng hoặc thu hẹp theo nhu cầu, hỗ trợ `auto-suspend` khi không hoạt động và áp dụng cơ chế tính phí theo thời gian sử dụng. **Storage** lưu dữ liệu tập trung theo định dạng **columnar** và tổ chức dữ liệu thành các `micro-partition`.

Nhờ tách biệt storage và compute, nhiều Virtual Warehouse có thể truy cập cùng một lớp lưu trữ mà không cạnh tranh tài nguyên tính toán. Kiến trúc này đồng thời cho phép chi phí lưu trữ và chi phí compute được quản lý độc lập.

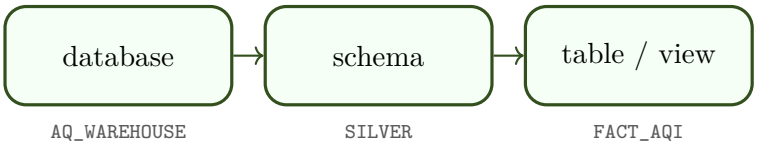
V.3. Core Objects

Trong Snowflake, dữ liệu được tổ chức theo một cấu trúc phân cấp gồm các **core objects**.

Database, Schema, Table, View

Database là vùng chứa logic cấp cao. **Schema** là **namespace** dùng để tổ chức các object bên trong database. **Table** lưu dữ liệu dưới dạng bảng, trong khi **View** biểu diễn dữ liệu dựa trên kết quả của một query.

Một tên object đầy đủ trong Snowflake có dạng:

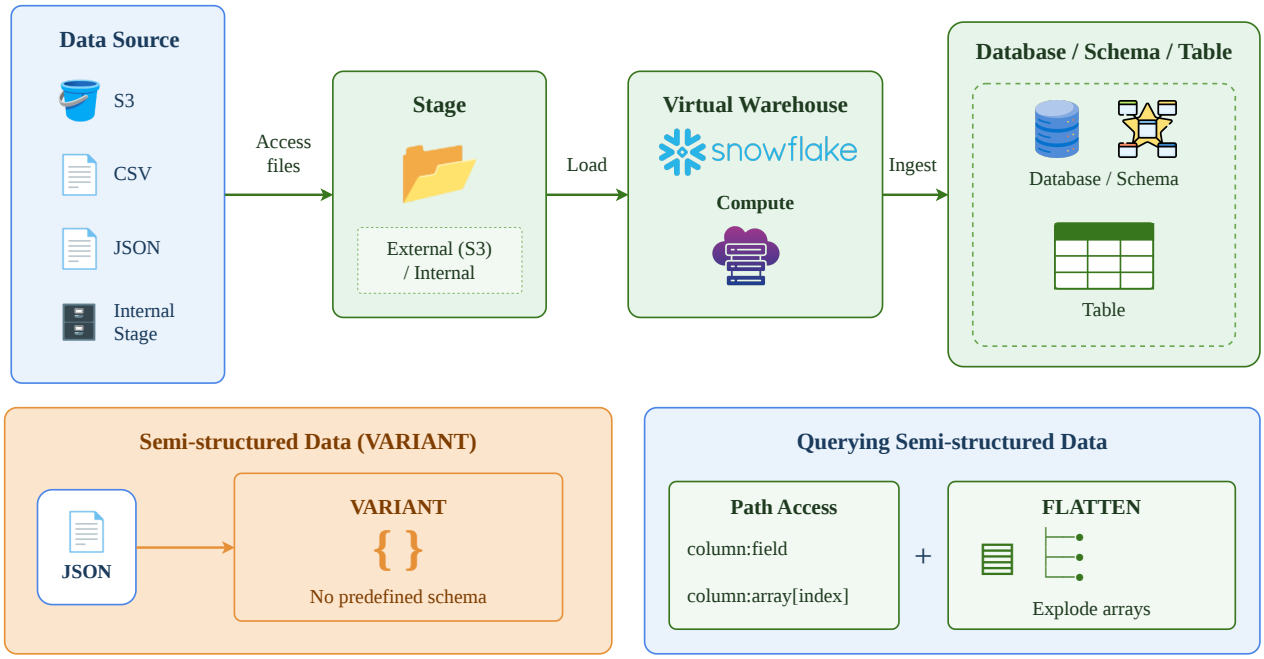


Hình 12: Tên object đầy đủ trong Snowflake theo cấu trúc `database.schema.table`.

V.4. Stage và Semi-structured Data

Snowflake cung cấp **Stage**, một object được sử dụng trong workflow xử lý tệp dữ liệu. Stage không phải là nơi lưu table đích, mà là đối tượng cho phép Snowflake truy cập vị trí của tệp nguồn, chẳng hạn tệp `CSV` hoặc `JSON` trên `S3`, hoặc tệp được lưu trong **internal stage**.

Một năng lực quan trọng của Snowflake là kiểu dữ liệu **VARIANT**, cho phép lưu **semi-structured data** như `JSON` mà không cần khai báo trước toàn bộ schema. Dữ liệu trong `VARIANT` vẫn có thể được truy vấn bằng SQL thông qua cú pháp đường dẫn `:` và hàm `FLATTEN`.



Hình 13: Workflow nạp dữ liệu. Dữ liệu nguồn được truy cập qua Stage và nạp vào Database/Schema/Table bằng compute của Virtual Warehouse.

V.5. Virtual Warehouse

Virtual Warehouse

Virtual Warehouse là tài nguyên compute dùng để thực thi workload trong Snowflake. Tài nguyên này có thể thay đổi quy mô, tự động tạm dừng khi không được sử dụng và được tính phí theo thời gian hoạt động.

Trong Snowflake, các câu lệnh đọc, biến đổi hoặc nạp dữ liệu được thực thi trên Virtual Warehouse. Warehouse được lựa chọn bằng lệnh `USE WAREHOUSE` và có thể được gắn với `resource monitor` nhằm giới hạn lượng credit tiêu thụ. Cơ chế này đặc biệt quan trọng đối với quản trị chi phí trong môi trường thực tế.

V.6. Các tính năng tích hợp quan trọng

Ngoài các core objects, Snowflake còn cung cấp nhiều tính năng tích hợp giúp giảm số lượng thành phần phải tự triển khai. **Time Travel** cho phép truy vấn trạng thái của bảng tại một thời điểm trong quá khứ hoặc khôi phục sau một lần nạp dữ liệu lỗi mà không cần duy trì snapshot riêng. **Zero-Copy Clone** tạo bản sao logic phục vụ phát triển mà không sao chép ngay toàn bộ dữ liệu lưu trữ. **Streams** hỗ trợ **Change Data Capture (CDC)** bằng cách theo dõi các thay đổi kể từ một offset xác định. **Tasks** cho phép lập lịch câu lệnh SQL và liên kết các tác vụ bằng quan hệ `AFTER` để hình thành **Directed Acyclic Graph (DAG)**. Bên cạnh đó, **micro-partition** và **clustering key** hỗ trợ giảm lượng dữ liệu cần quét khi thực thi query. Các tính năng này sẽ được minh họa cụ thể trong dự án ở phần tiếp theo.

VI. Project: Vietnam Air Quality Data Pipeline using Snowflake

VI.1. Business Requirement & Multi-source Data Source

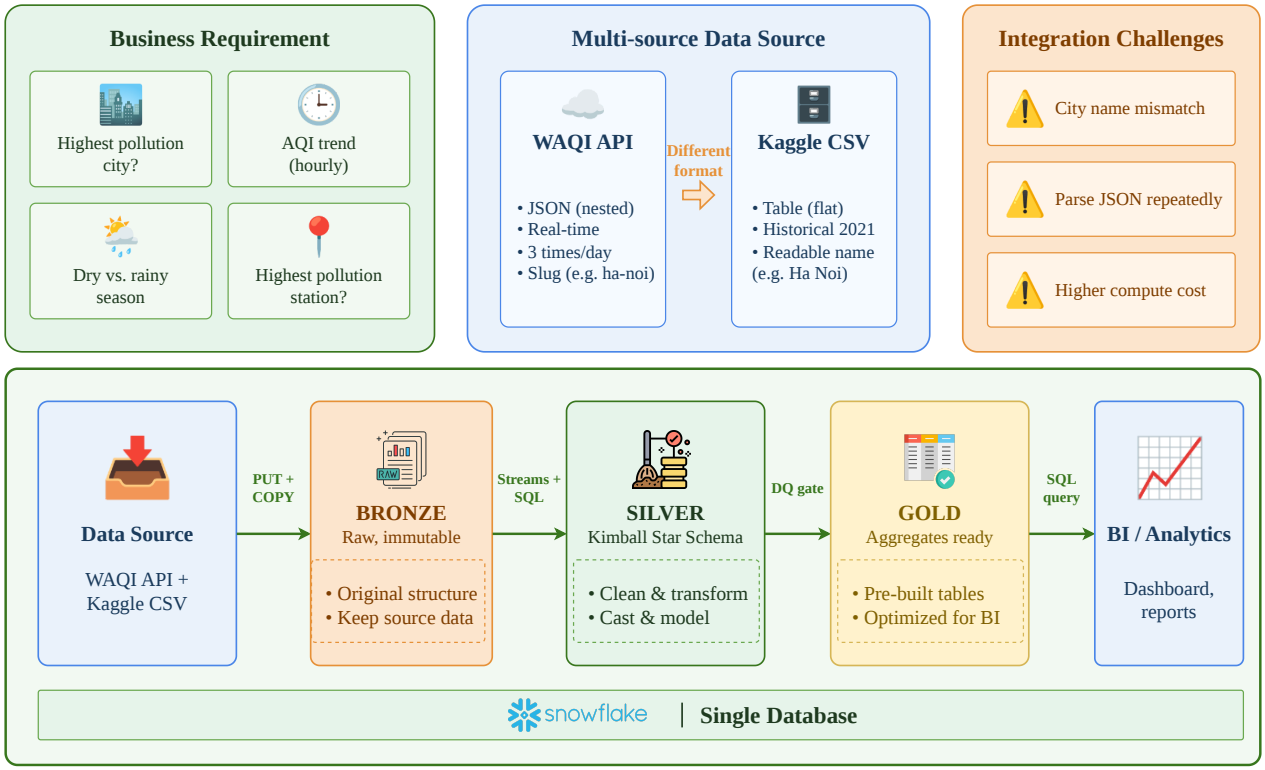
Dự án được xây dựng nhằm hỗ trợ các câu hỏi phân tích như: thành phố nào trong năm thành phố được theo dõi có mức ô nhiễm cao nhất? AQI biến động như thế nào theo giờ? Chất lượng không khí khác biệt ra sao giữa mùa khô và mùa mưa? Và station nào ghi nhận mức ô nhiễm cao nhất trong cùng một thành phố?

Dữ liệu được thu thập từ hai nguồn có cấu trúc khác nhau. **WAQI API** trả về dữ liệu `JSON` lồng nhau theo thời gian thực, với tần suất ba lần mỗi ngày cho từng thành phố. Trong khi đó, **Kaggle CSV** cung cấp dữ liệu lịch sử năm 2021 dưới dạng bảng phẳng và sử dụng tên thành phố dễ đọc như `Ha Noi`; WAQI API lại sử dụng **slug** như `ha-noi`.

Sự khác biệt giữa hai nguồn đặt ra yêu cầu tích hợp ngay từ giai đoạn thiết kế. Nếu dữ liệu chỉ được đưa vào cùng một kho lưu trữ mà không qua chuẩn hóa, quá trình phân tích sẽ gặp nhiều trở ngại: tên thành phố không tương thích, cấu trúc `JSON` phải được phân rã lặp lại trong từng truy vấn, và việc tái xử lý toàn bộ dữ liệu làm gia tăng chi phí compute.

VI.2. Kiến trúc Medallion

Dự án áp dụng **Medallion Architecture**, trong đó dữ liệu được tổ chức thành ba lớp `Bronze`, `Silver` và `Gold`; mỗi lớp đảm nhiệm một mục tiêu xử lý riêng. Cả ba lớp được triển khai trong cùng một Snowflake database.

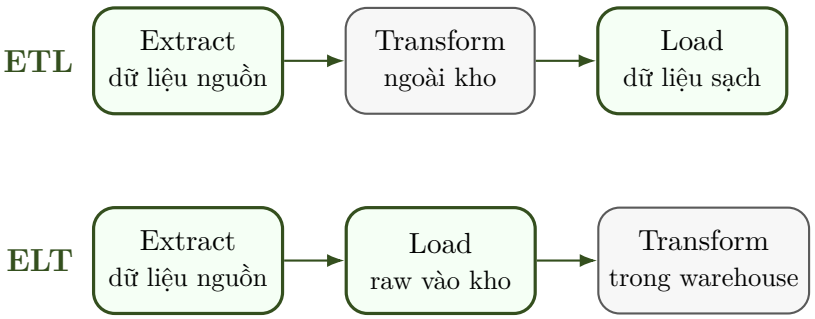


Hình 14: Luồng xử lý tổng quát. Dữ liệu đi từ nguồn qua ba lớp Bronze, Silver và Gold trước khi được cung cấp cho công cụ BI.

Bronze bảo toàn dữ liệu thô theo cấu trúc của nguồn. **Silver** thực hiện làm sạch, ép kiểu và mô hình hóa dữ liệu thành Star Schema. **Gold** cung cấp các bảng **aggregate** đã được chuẩn bị cho **Business Intelligence (BI)**.

VI.3. ELT thay vì ETL

Pipeline sử dụng mô hình **Extract–Load–Transform (ELT)** thay cho **Extract–Transform–Load (ETL)** truyền thống. Với ETL, dữ liệu được biến đổi trước khi nạp vào kho, do đó cấu trúc đích phải được xác định từ giai đoạn đầu. Với ELT, dữ liệu thô được nạp trước và các phép biến đổi được thực hiện sau đó bằng năng lực compute của warehouse.



Hình 15: So sánh ETL và ELT. Trong ELT, raw data được nạp vào warehouse trước, bước Transform được thực thi bên trong kho và có thể được tái thực hiện khi cần.

VI.4. Lớp CONTROL và bài toán conformance

Một yêu cầu quan trọng của quá trình tích hợp là **conformance** giữa các nguồn dữ liệu khi cùng mô tả một thành phố. Nguồn CSV sử dụng tên dễ đọc như `Ha Noi` hoặc `Ho Chi Minh City`, trong khi API sử dụng slug như `ha-noi`. Nếu không chuẩn hóa, cùng một thành phố có thể bị nhận diện thành nhiều thực thể khác nhau.

Yêu cầu này được xử lý bằng hai bảng tham chiếu trong schema `CONTROL`. Bảng `CITY_MAP` ánh xạ các biến thể tên thành phố sang slug chuẩn. Bảng `CITY_REFERENCE` liên kết mỗi slug với tên hiển thị, vùng địa lý và tọa độ trung tâm.

Tên trong CSV	slug chuẩn
Ha Noi / Hanoi	ha-noi
Ho Chi Minh City / Ho Chi Minh	ho-chi-minh-city
Da Nang	da-nang
Gia Lai	gia-lai
Cao Bang	cao-bang

Trong môi trường thực tế, API yêu cầu `token` và kết nối mạng. Vì vậy, dự án cung cấp thêm **seed data** được sinh theo cơ chế **deterministic**, qua đó cho phép toàn bộ pipeline chạy ở chế độ `offline` mà không phụ thuộc vào token hoặc kết nối mạng.

VI.5. Bronze lưu giữ nguyên dữ liệu thô

Lớp `Bronze` hiện thực hóa nguyên tắc ELT bằng cách lưu dữ liệu nguồn ở trạng thái nguyên bản và không sửa đổi. Thiết kế này bảo toàn khả năng truy vết và cho phép tái xử lý các lớp phía trên khi logic biến đổi thay đổi.

Tại lớp này, kiểu `VARIANT` của Snowflake cho phép một cột lưu toàn bộ **JSON payload** trong khi dữ liệu vẫn có thể được truy vấn bằng SQL.

```sql
CREATE OR REPLACE TABLE BRONZE.API_RAW (
    queried_city  STRING  NOT NULL COMMENT 'Thanh pho da truy van',
    raw_payload   VARIANT NOT NULL COMMENT 'Toan bo JSON goc',
    ingested_at   TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
    source_file   STRING
);
```

Câu truy vấn dưới đây trích xuất trực tiếp các trường từ `JSON` mà không cần định nghĩa trước toàn bộ schema:

```sql
SELECT
    raw_payload:data:aqi::DOUBLE        AS aqi,
    raw_payload:data:city:name::STRING  AS station
FROM BRONZE.API_RAW
```

```
LIMIT 10;
```

Bảng BRONZE.HISTORICAL_CSV cũng tuân theo nguyên tắc bảo toàn dữ liệu thô bằng cách giữ toàn bộ cột ở kiểu STRING. Lớp Bronze không áp dụng quy tắc chất lượng hoặc diễn giải nghiệp vụ, mà ghi nhận dữ liệu đúng theo cấu trúc nguồn. Việc chuyển đổi từ STRING sang DOUBLE hoặc TIMESTAMP thuộc trách nhiệm của lớp Silver.

VI.6. Load (bước nạp dữ liệu vào Snowflake)

Bước **Load** chỉ thực hiện di chuyển dữ liệu và không áp dụng biến đổi nghiệp vụ. Quy trình gồm hai thao tác: PUT tải tệp cục bộ lên internal stage, sau đó COPY INTO đọc tệp từ stage và ghi dữ liệu vào bảng thuộc lớp Bronze.

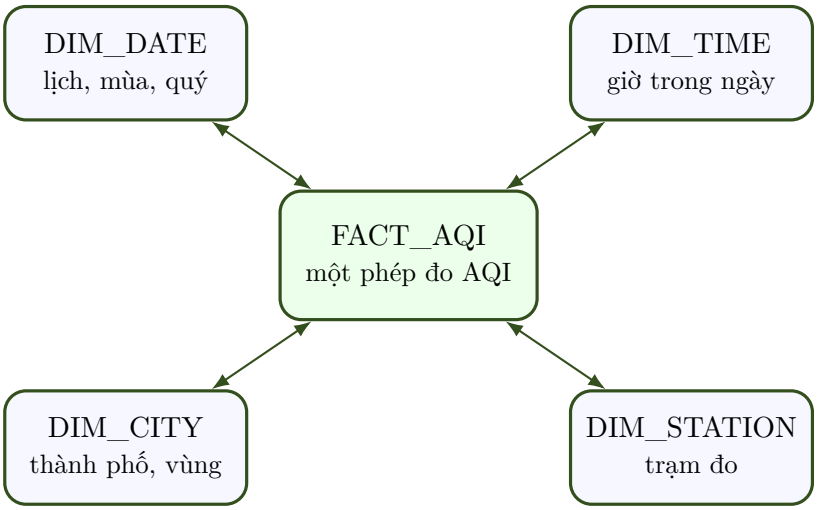
```
COPY INTO BRONZE.API_RAW (queried_city, raw_payload, source_file)
FROM (
    SELECT
      $1:data:_queried_city::STRING AS queried_city,
      $1                            AS raw_payload,
      METADATA$FILENAME             AS source_file
    FROM @BRONZE.AQ_INTERNAL_STAGE
)
PATTERN = '.*[.]json'
FILE_FORMAT = BRONZE.AQ_JSON_FORMAT
ON_ERROR = 'CONTINUE';
```

Để ngăn một tệp được nạp nhiều lần khi pipeline được tái thực thi, dự án sử dụng bảng CONTROL.LOADED_FILES như một cơ chế bảo đảm **idempotency**. Mỗi tệp đã xử lý được ghi nhận, nhờ đó các lần chạy sau có thể bỏ qua tệp tương ứng.

VI.7. Silver được tổ chức theo mô hình Kimball

Dữ liệu trong Bronze đã có thể truy vấn nhưng chưa phù hợp để sử dụng trực tiếp cho analytics. Đối với yêu cầu như xác định thành phố có mức ô nhiễm cao nhất trong mùa khô, người viết truy vấn sẽ phải lặp lại các thao tác phân rã JSON, liên kết định nghĩa mùa và ánh xạ tên thành phố trước khi thực hiện phép tính.

Lớp Silver tổ chức lại dữ liệu theo **Kimball Star Schema**. Fact Table ở trung tâm lưu các measurement, còn các Dimension Table xung quanh lưu thuộc tính mô tả. Trong dự án, Fact Table là FACT_AQI, bốn Dimension Table gồm DIM_DATE, DIM_TIME, DIM_CITY và DIM_STATION.



Hình 16: Mô hình Star Schema của lớp Silver. Fact table ở trung tâm tham chiếu bốn Dimension Table thông qua surrogate key.

Thiết kế sử dụng **surrogate key**: thay vì dùng khóa tự nhiên như slug của thành phố, mỗi Dimension được gán một khóa số do `sequence` sinh ra và không mang ý nghĩa nghiệp vụ. Khóa tự nhiên có thể thay đổi theo source system, trong khi surrogate key độc lập với nguồn và duy trì ổn định trong warehouse. Riêng `DIM_DATE` và `DIM_TIME` sử dụng **smart key** lần lượt là `YYYYMMDD` và `HHMM`.

Đoạn mã dưới đây trình bày logic chính của **stored procedure** `SILVER.LOAD_FROM_BRONZE()` gồm thu nhận dữ liệu thay đổi từ các Stream, lưu vào bảng tạm, đồng bộ Dimension và chèn dữ liệu vào `FACT_AQI`.

```
-- 1. Gom du lieu moi tu stream vao bang tam
CREATE OR REPLACE TEMPORARY TABLE _SRC_MEASUREMENT AS
SELECT
    r.raw_payload:data:idx::INT      AS waqi_idx,
    r.raw_payload:data:aqi::DOUBLE   AS aqi,
    r.queried_city                   AS city_slug,
    'api'                            AS source,
    ...
FROM BRONZE.API_RAW_STREAM r
UNION ALL
SELECT ... FROM BRONZE.HISTORICAL_CSV_STREAM c;

-- 2. Dong bo dimension thanh pho (idempotent MERGE)
MERGE INTO SILVER.DIM_CITY AS t
USING CONTROL.CITY_REFERENCE AS s
ON t.city_slug = s.city_slug
WHEN MATCHED THEN UPDATE SET t.city_name = s.city_name, ...
WHEN NOT MATCHED THEN INSERT (city_key, city_slug, ...)
    VALUES (SILVER.SEQ_CITY_KEY.NEXTVAL, s.city_slug, ...);

-- 3. Chen vao fact, noi voi ca bon dimension
INSERT INTO SILVER.FACT_AQI (date_key, time_key, city_key, station_key,
    ...)
```

```
SELECT d.date_key, tm.time_key, c.city_key, st.station_key, s.aqi, ...
FROM _SRC_MEASUREMENT s
JOIN SILVER.DIM_CITY c      ON c.city_slug = s.city_slug
JOIN SILVER.DIM_STATION st  ON st.waqi_idx = s.waqi_idx AND st.city_key =
    c.city_key
JOIN SILVER.DIM_DATE d      ON d.full_date = s.measured_at::DATE
JOIN SILVER.DIM_TIME tm     ON tm.hour = HOUR(s.measured_at);
```

Việc sử dụng `MERGE` thay cho `INSERT` đơn thuần đối với các Dimension giúp thao tác đồng bộ có tính idempotent: việc tái thực thi không tạo thêm bản ghi trùng lặp.

VI.8. Streams, Tasks và cơ chế Orchestration tích hợp

Việc tái thực thi toàn bộ quy trình mỗi khi có dữ liệu mới làm gia tăng chi phí compute và thường đòi hỏi một hệ thống điều phối bên ngoài. Snowflake cung cấp hai cơ chế tích hợp để xử lý yêu cầu này ngay trong data warehouse.

Cơ chế thứ nhất là **Stream**, một đối tượng hỗ trợ Change Data Capture bằng cách theo dõi các thay đổi dữ liệu kể từ một offset đã được ghi nhận.

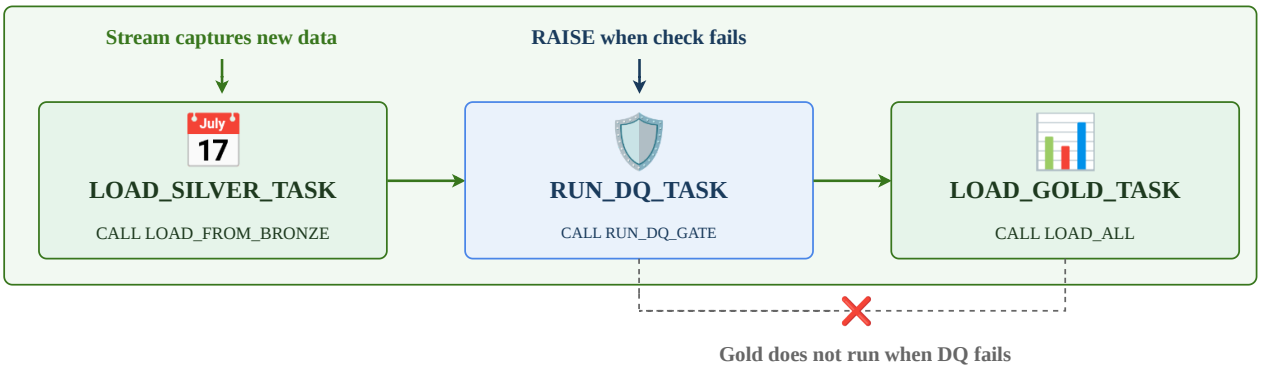
```
CREATE OR REPLACE STREAM BRONZE.API_RAW_STREAM
  ON TABLE BRONZE.API_RAW
  APPEND_ONLY = TRUE;
```

Cơ chế thứ hai là **Task**, cho phép lập lịch thực thi câu lệnh SQL trên warehouse. Các Task có thể được liên kết bằng quan hệ `AFTER` để hình thành một DAG tuần tự mà không cần bộ lập lịch bên ngoài.

```
CREATE OR REPLACE TASK CONTROL.LOAD_SILVER_TASK
  WAREHOUSE = AQ_WH
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('BRONZE.API_RAW_STREAM')
      OR SYSTEM$STREAM_HAS_DATA('BRONZE.HISTORICAL_CSV_STREAM')
AS
  CALL SILVER.LOAD_FROM_BRONZE();

CREATE OR REPLACE TASK CONTROL.RUN_DQ_TASK
  WAREHOUSE = AQ_WH
  AFTER CONTROL.LOAD_SILVER_TASK
AS
  CALL CONTROL.RUN_DQ_GATE();

CREATE OR REPLACE TASK CONTROL.LOAD_GOLD_TASK
  WAREHOUSE = AQ_WH
  AFTER CONTROL.RUN_DQ_TASK
AS
  CALL GOLD.LOAD_ALL();
```



Hình 17: Task DAG trong Snowflake. Khi Data Quality Gate phát sinh lỗi bằng lệnh `RAISE`, Task nạp lớp Gold ở phía sau không được thực thi.

VI.9. Data Quality Gate giúp ngăn dữ liệu không đạt yêu cầu

Sau khi dữ liệu được mô hình hóa trong `Silver`, chất lượng dữ liệu cần được xác minh trước khi xây dựng lớp `Gold`. Toàn bộ logic được đóng gói trong stored procedure `CONTROL.RUN_DQ_GATE()`. Procedure thực hiện bảy phép kiểm tra, ghi kết quả vào bảng `CONTROL.DQ_RESULTS` và sử dụng `RAISE` để phát sinh ngoại lệ nếu có bất kỳ kiểm tra nào thất bại.

```
IF (v_failures > 0) THEN
    RAISE DQ_FAILURE;
END IF;
RETURN 'DQ PASSED';
```

Bảng dưới đây trình bày bảy phép kiểm tra và mục tiêu tương ứng.

Check	Ý nghĩa
`row_count`	bảo đảm số dòng tối thiểu cho phân tích
`null_pct`	bảo đảm tỷ lệ null trên các cột quan trọng dưới ngưỡng cho phép
`aqi_range`	AQI nằm trong khoảng hợp lệ `[0, 500]`
`city_coverage`	mọi thành phố tham chiếu đều xuất hiện trong Fact Table
`source_validity`	chỉ chấp nhận nguồn `kaggle` hoặc `api`
`freshness`	có ít nhất một dòng được nạp trong 48 giờ gần nhất
`fk_integrity`	mọi dòng Fact đều liên kết được với cả bốn Dimension

Lệnh `RAISE` tương tác trực tiếp với Task DAG. Khi `RUN_DQ_TASK` gọi procedure và procedure phát sinh ngoại lệ, Task được đánh dấu thất bại; do đó, Task phụ thuộc được khai báo bằng `AFTER`, tức `LOAD_GOLD_TASK`, không được thực thi. Cơ chế này hiện thực hóa hành vi **pipeline fail-fast** hoàn toàn bằng các thành phần tích hợp của Snowflake.

VI.10. Gold lưu dữ liệu tổng hợp phục vụ BI

Lớp `Silver` cung cấp dữ liệu đã được làm sạch và chuẩn hóa nhưng vẫn duy trì grain ở cấp từng phép đo. Đối với dashboard biểu diễn xu hướng theo tháng, việc đọc lại hàng nghìn dòng Fact trong mỗi truy vấn là không cần thiết. Lớp `Gold` giải quyết yêu cầu này bằng cách cung cấp dữ liệu đã được tổng hợp trước.

Lớp `Gold` không diễn giải lại dữ liệu nguồn mà được xây dựng từ các bảng đã chuẩn hóa trong `Silver`. Nhờ đó, các Data Mart ở Gold tái sử dụng nhất quán các định nghĩa nghiệp vụ đã được thiết lập tại Silver. Dự án tạo năm bảng aggregate: `AQI_DAILY_SUMMARY`, `AQI_CITY_RANKING`, `STATION_SUMMARY`, `AQI_HOURLY_PATTERN` và `AQI_SEASONAL_SUMMARY`.

```sql
WITH daily AS (
    SELECT
        c.city_slug,
        d.full_date AS date,
        AVG(f.aqi)  AS avg_aqi,
        MAX(f.aqi)  AS max_aqi
    FROM SILVER.FACT_AQI f
    JOIN SILVER.DIM_CITY c ON c.city_key = f.city_key
    JOIN SILVER.DIM_DATE d ON d.date_key = f.date_key
    GROUP BY c.city_slug, d.full_date
),
monthly AS (
    SELECT
        city_slug,
        YEAR(date)  AS year,
        MONTH(date) AS month,
        AVG(avg_aqi) AS avg_aqi
    FROM daily
    GROUP BY city_slug, YEAR(date), MONTH(date)
)
SELECT
    city_slug, year, month, avg_aqi,
    DENSE_RANK() OVER (ORDER BY avg_aqi DESC) AS aqi_rank
FROM monthly;
```

Năm procedure thành phần được điều phối bởi procedure `GOLD.LOAD_ALL()`, qua đó cung cấp một điểm gọi thống nhất cho cả client cục bộ và Task.

VI.11. Lựa chọn Snowflake và kiểm soát chi phí

Việc lựa chọn Snowflake thay cho kiến trúc AWS ban đầu không hàm ý rằng Snowflake ưu việt trong mọi bối cảnh. Đối với phạm vi dự án này, lợi ích chính là giảm số lượng dịch vụ độc lập cần triển khai, tích hợp và vận hành.

Bảng 4: Ánh xạ các dịch vụ trong kiến trúc AWS ban đầu sang cơ chế tương ứng trong Snowflake.

AWS service	Vai trò	Snowflake equivalent
S3	object storage	bảng trong warehouse
Glue Crawler + Catalog	khám phá schema	VARIANT + FLATTEN
Glue Jobs (PySpark)	biến đổi	SQL stored procedure
Lambda	gọi API, kiểm tra chất lượng	stored procedure
Step Functions	điều phối	Task DAG (AFTER)
EventBridge	lên lịch	Task SCHEDULE
Athena	truy vấn	cùng một SQL engine
SNS	cảnh báo	DQ_RESULTS + RAISE

Đối với một dự án giáo dục, **kiểm soát chi phí** là một yêu cầu thiết kế quan trọng. Snowflake cho phép gắn **resource monitor** với warehouse để thiết lập hạn mức credit tiêu thụ theo tháng.

```
CREATE RESOURCE MONITOR AQ_BUDGET_MONITOR
  WITH CREDIT_QUOTA = 400 FREQUENCY = MONTHLY
  TRIGGERS
    ON 75 PERCENT  DO NOTIFY
    ON 90 PERCENT  DO NOTIFY
    ON 100 PERCENT DO SUSPEND_IMMEDIATE;
```

Resource monitor không cải thiện trực tiếp hiệu năng pipeline, nhưng giúp hạn chế rủi ro chi phí phát sinh từ lỗi lập lịch hoặc từ việc warehouse tiếp tục hoạt động ngoài dự kiến.

VII. Kết luận

Một data platform hiệu quả là kết quả của các quyết định kiến trúc có cơ sở, không chỉ là tập hợp các câu lệnh có thể thực thi. Sự khác biệt giữa analytical workload và operational workload cho thấy nhu cầu tách data warehouse khỏi hệ thống vận hành.

Data warehouse có thể được phát triển theo hướng Inmon với nguyên tắc **enterprise-first** hoặc Kimball với nguyên tắc **business-process-first**. Star Schema được hình thành từ yêu cầu analytics thông qua Business Process, grain, Dimensions và Facts, thay vì từ việc phân chia bảng tùy ý.

Snowflake hiện thực hóa kiến trúc này bằng mô hình tách biệt storage và compute, kiểu `VARIANT` cho semi-structured data, cùng các tính năng tích hợp như `Streams`, `Tasks`, `Time Travel` và `resource monitor`.

Dự án Vietnam Air Quality Data Pipeline hiện thực hóa kiến trúc này trong một quy trình thống nhất: schema `CONTROL` chuẩn hóa định danh thành phố; `Bronze` bảo toàn dữ liệu thô; `Silver` tổ chức Kimball Star Schema; `Streams` và `Tasks` thực hiện orchestration; Data Quality Gate kiểm soát chất lượng; và `Gold` cung cấp dữ liệu tổng hợp cho BI. Qua đó, dự án minh họa cả quy trình triển khai và cơ sở phương pháp cho các data warehouse có yêu cầu tương tự. Tham khảo reposetory hoàn chỉnh ở link sau: Vietnam-Air-Quality-Data-Pipeline-Snowflake

VIII. Tài liệu tham khảo

[1] W. H. Inmon, *Building the Data Warehouse*, 4th ed. Indianapolis, IN, USA: Wiley, 2005, ISBN: 978-0-7645-9944-6.

[2] R. Kimball et al., *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2013, ISBN: 978-1-118-53080-1.

[3] Snowflake Inc. "Snowflake key concepts and architecture". (), [Online]. Available: https://docs.snowflake.com/en/user-guide/intro-key-concepts (visited on 09/18/2026).

[4] Snowflake Inc. "Introduction to loading semi-structured data". (), [Online]. Available: https://docs.snowflake.com/en/user-guide/semistructured-intro (visited on 09/18/2026).

[5] Snowflake Inc. "Understanding and using time travel". (), [Online]. Available: https://docs.snowflake.com/en/user-guide/data-time-travel (visited on 09/18/2026).

[6] Snowflake Inc. "Introduction to streams". (), [Online]. Available: https://docs.snowflake.com/en/user-guide/streams-intro (visited on 09/18/2026).

[7] Snowflake Inc. "Introduction to tasks". (), [Online]. Available: https://docs.snowflake.com/en/user-guide/tasks-intro (visited on 09/18/2026).

[8] Snowflake Inc. "Working with resource monitors". (), [Online]. Available: https://docs.snowflake.com/en/user-guide/resource-monitors (visited on 09/18/2026).

[9] Databricks Inc. "What is the medallion lakehouse architecture?" (), [Online]. Available: https://docs.databricks.com/aws/en/lakehouse/medallion (visited on 09/18/2026).

[10] World Air Quality Index Project. "Api – air quality programmatic apis". (), [Online]. Available: https://aqicn.org/api/ (visited on 09/18/2026).

[11] dangnha. "Vietnam air quality – snowflake-native data warehouse". (), [Online]. Available: https://github.com/dangnha/Vietnam-Air-Quality-Data-Pipeline-Snowflake (visited on 09/18/2026).

Phụ lục

1. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây.

